AI-Driven Development with Claude

Spec-driven development, hands-on: from a stakeholder request to tested code, with Claude Code and Wavez skills

Specs › Implementation › Existing feature

Lab · about 3 hours

https://tinyurl.com/zenika0311

Our Zenika team for today



Michael Isvy

Head of Engineering



Asyraaf Ahmad

Tech Lead



Truong Mai Gia

Senior Consultant



Martin Llaneza

Senior Consultant



Xavier Jamito

Consultant

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Today’s flow

1

Get started

Everyone

Clone, run and explore the app

2

Specs

Product manager

Brief › PRD › prototype › PRD v2

3

Implementation

Developer, then QA

Backend › frontend › review › UAT

4

Existing feature

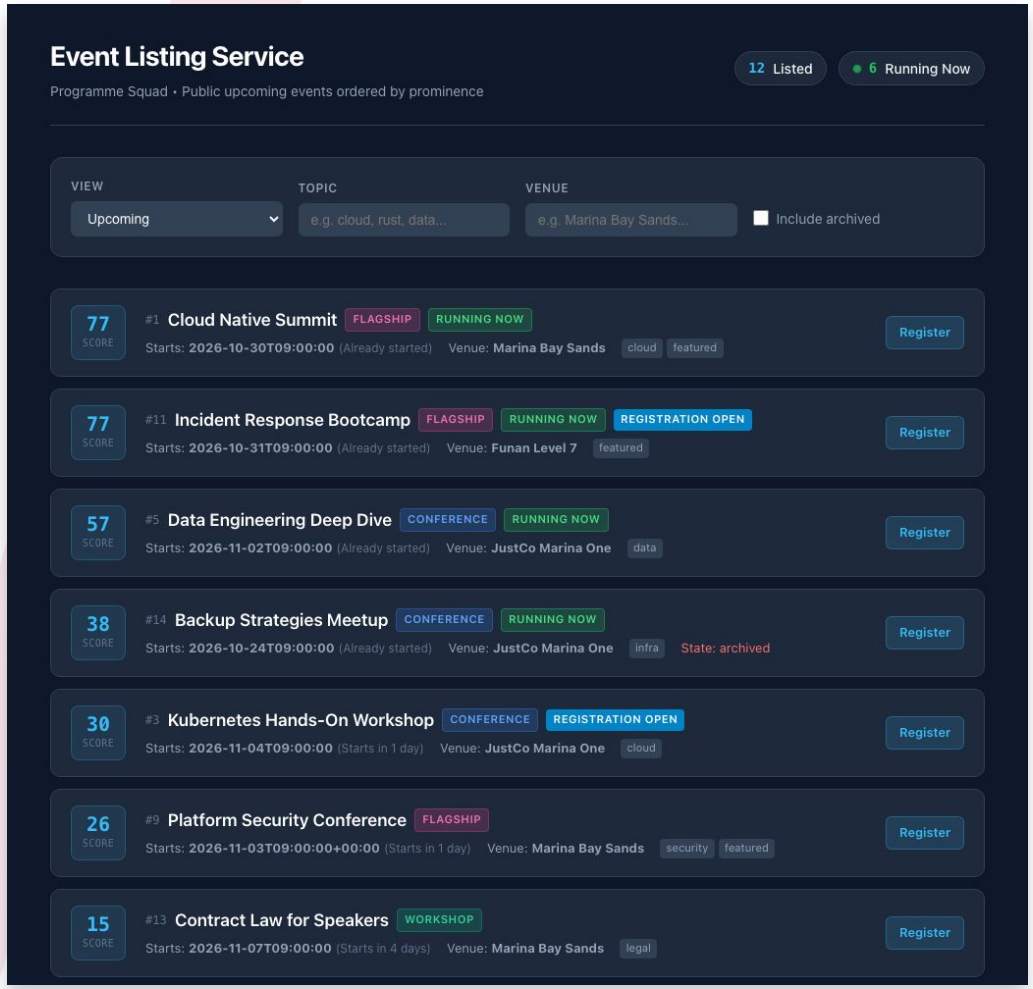
PM, then developer

Spec from the code › same flow as 3

Each step writes a file. The next step reads it.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Your app: the Event Listing Application



Lists upcoming tech events, ranked by a prominence score

Next.js, React, TypeScript, SQLite

Your team just inherited it. Nobody on the team wrote it.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Two jobs waiting for you



NEW FEATURE · SECTIONS 2 AND 3

Add and edit events

- Today an engineer types events into the database
- 2 to 3 days per change



EXISTING FEATURE · SECTION 4, OPTIONAL

Fix the listing page

- Attendees find the page confusing
- Search misses the events they expect

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

One repo, two workspaces

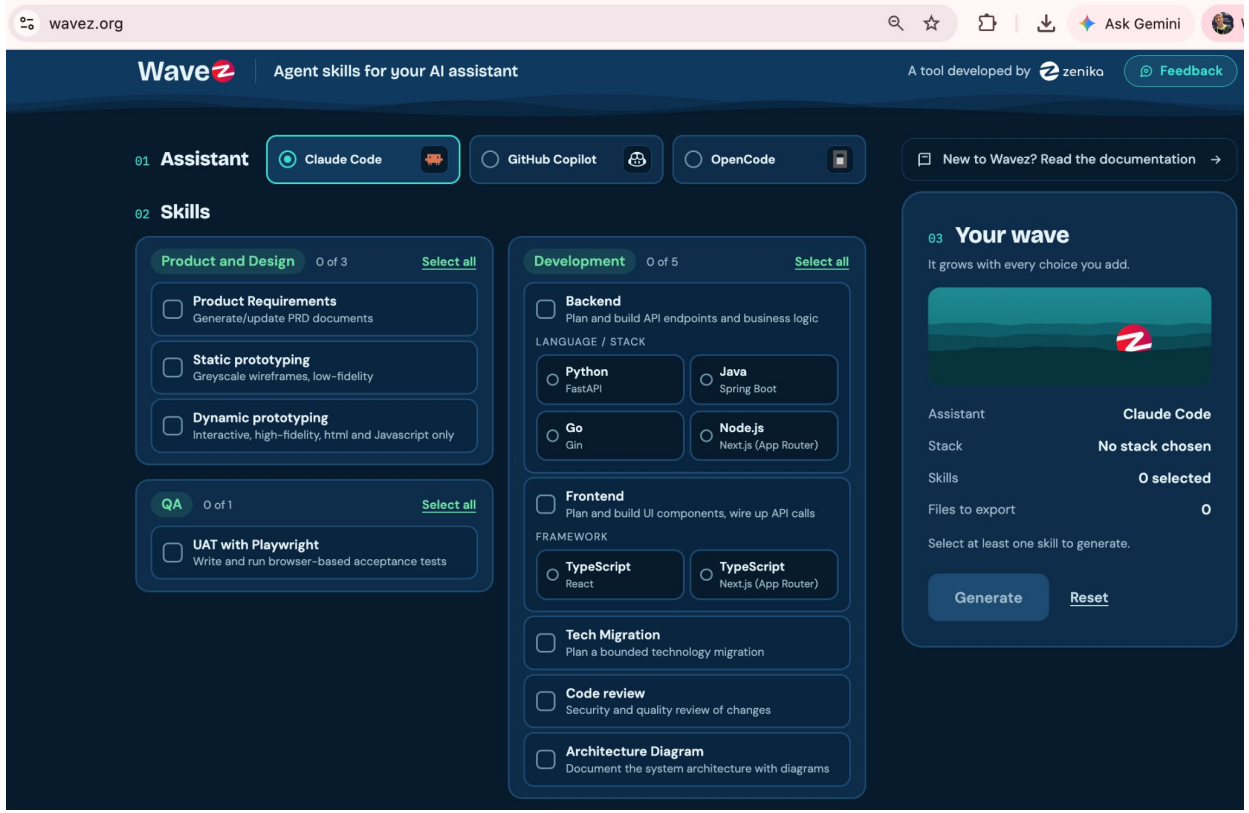
```
sg-stack-workshop/
├── 01-product-management/          Section 2
└── 02-application/
    └── event-listing-service/      Sections 1, 3 and 4
```

One VS Code window per workspace.
Start claude in that folder, or its skills won't show up.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Managing your skills with Wavez

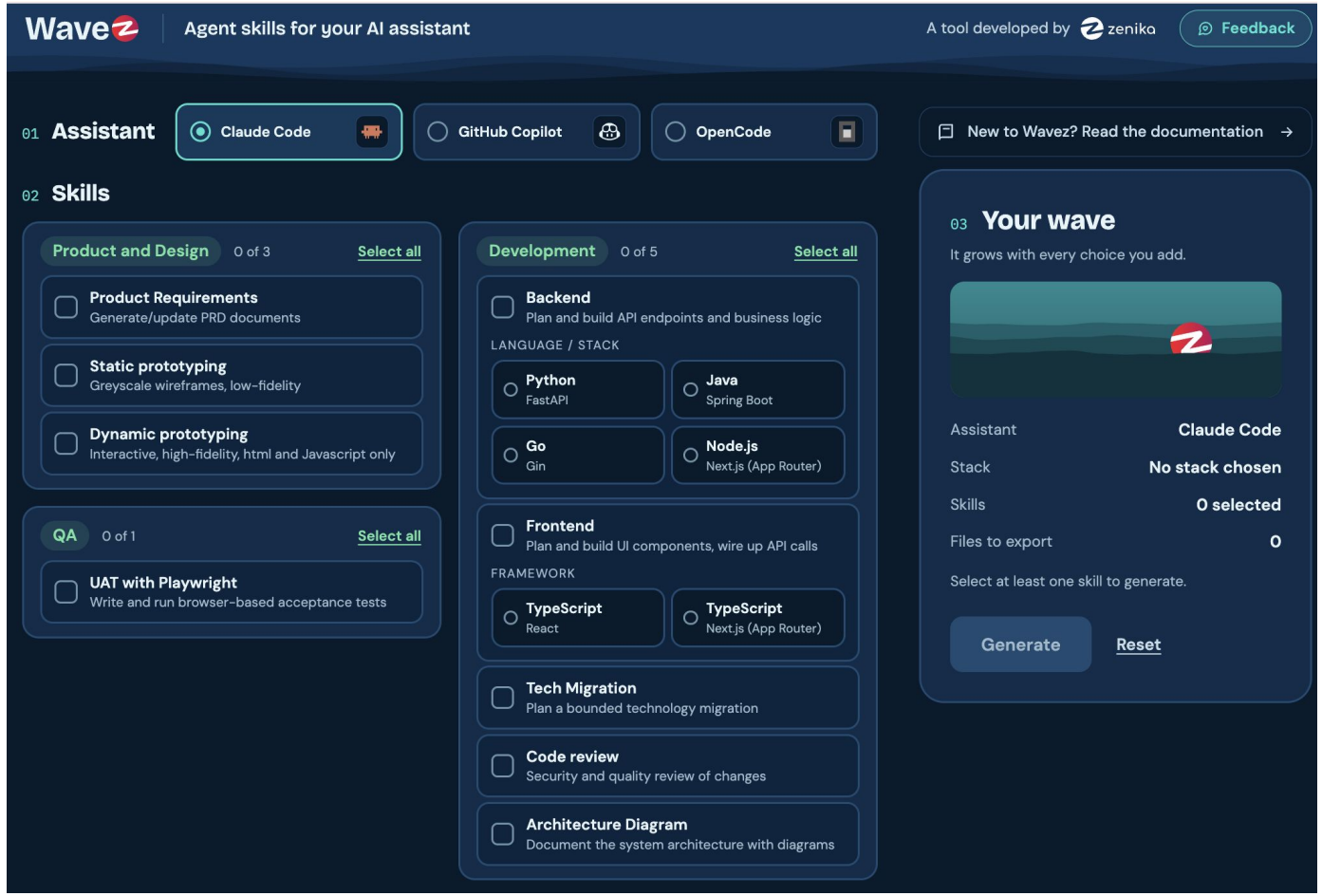
- Online Portal for Skills generation
- Covers both Product Discovery and Engineering
- Free and Open Source
- wavez.org



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Wavez in 3 steps

1. Select your Assistant
2. Select your skills
3. Generate your project and download the zip file to be imported into your project

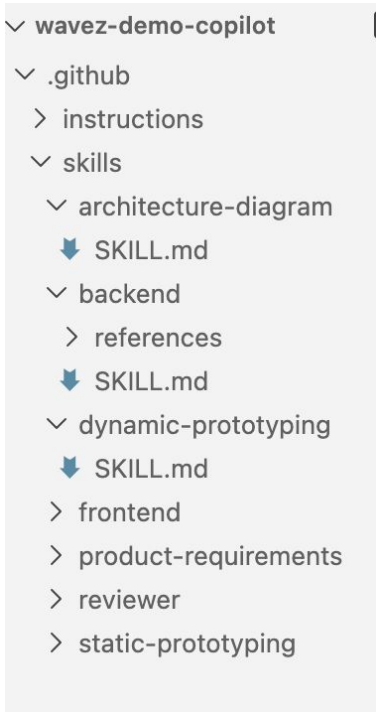


© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

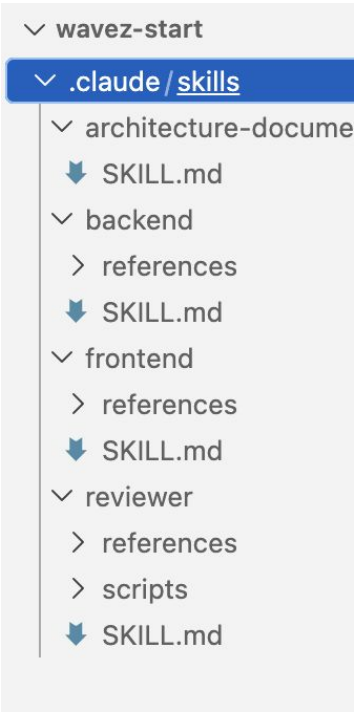
Importing your skills inside VS Code

1. Unzip your file
2. Copy/paste at the root of the workspace

Example with Github Copilot



Example with Claude code



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

1

Get started

10 min · run and explore the app

WATCH OUT

- Keep npm run dev running: that terminal is your app terminal

1.1 Clone the repo

1.2 npm install, npm test, npm run dev

1.3 Explore localhost:3000

2

Specs

45 min · you are the product manager

WATCH OUT

- /clear before each step
- The PRD skill asks questions. If the brief has no answer, you decide.

2.1 Open 01-product-management

2.2 Wavez download 1

2.3 Requirements brief

2.4 PRD

2.5 Clickable prototype

2.6 Feedback, then PRD v2

3

Implementation

85 min · developer, then QA

WATCH OUT

- Three terminals: app, Claude, commands
- /reviewer, not the built-in /code-review
- No npm test during UAT: it resets the database
- No PRD? Ask for the reference PRD

3.1 Back to the app window

3.2 Wavez download 2

3.3 Copy the PRD, gap analysis

3.4 Backend

3.5 Frontend

3.6 Review

3.7 Acceptance tests (UAT)

4

Existing feature

45 min · optional

WATCH OUT

- 4.1 changes no code: spec first
- Rerun the Section 3 test plan to catch regressions

4.1 Write the spec from the code

4.2 Backend, frontend, review

4.3 UAT, plus the Section 3 tests

Feedback



https://go.gov.sg/stack2026-workshop
feedback

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Feedback



https://go.gov.sg/stack2026-workshop
feedback

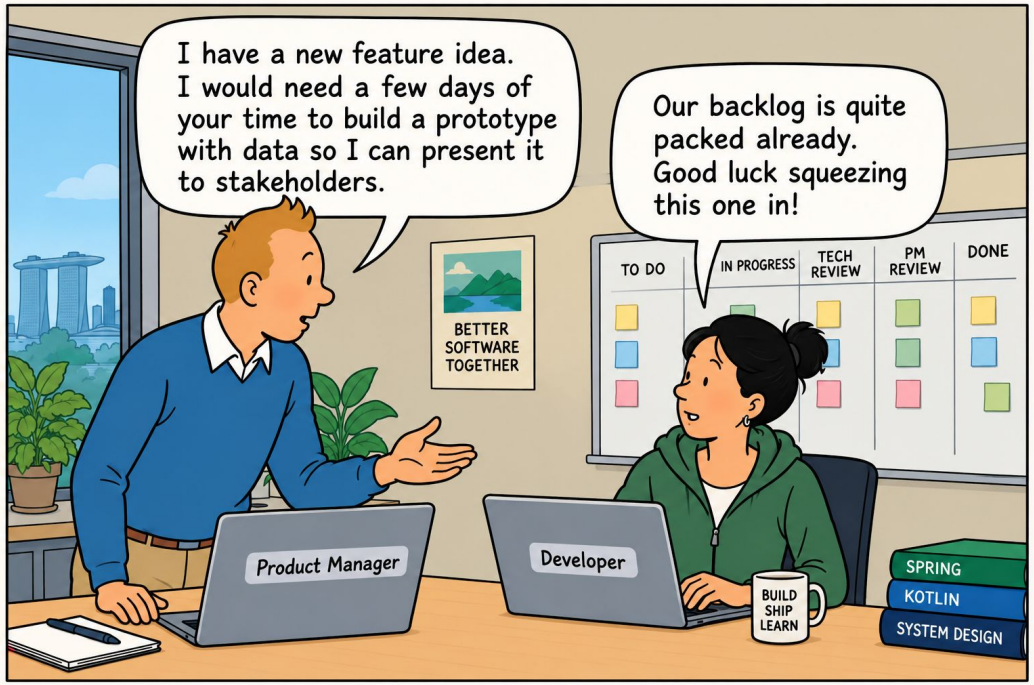
Appendix



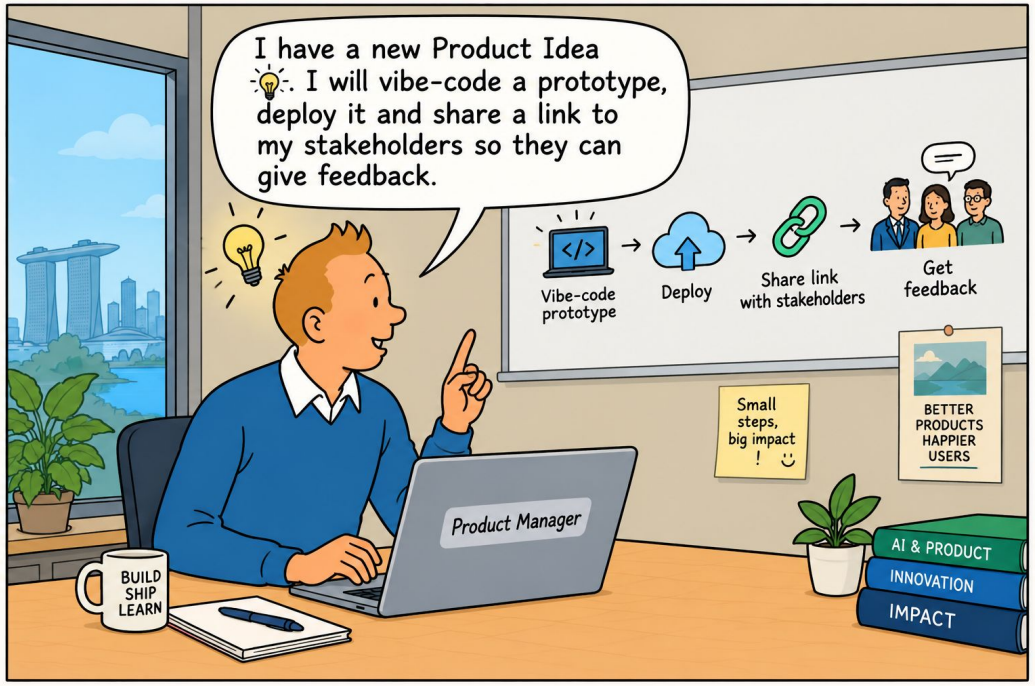
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Product Manager needs to validate an idea

In 2023 (without AI)



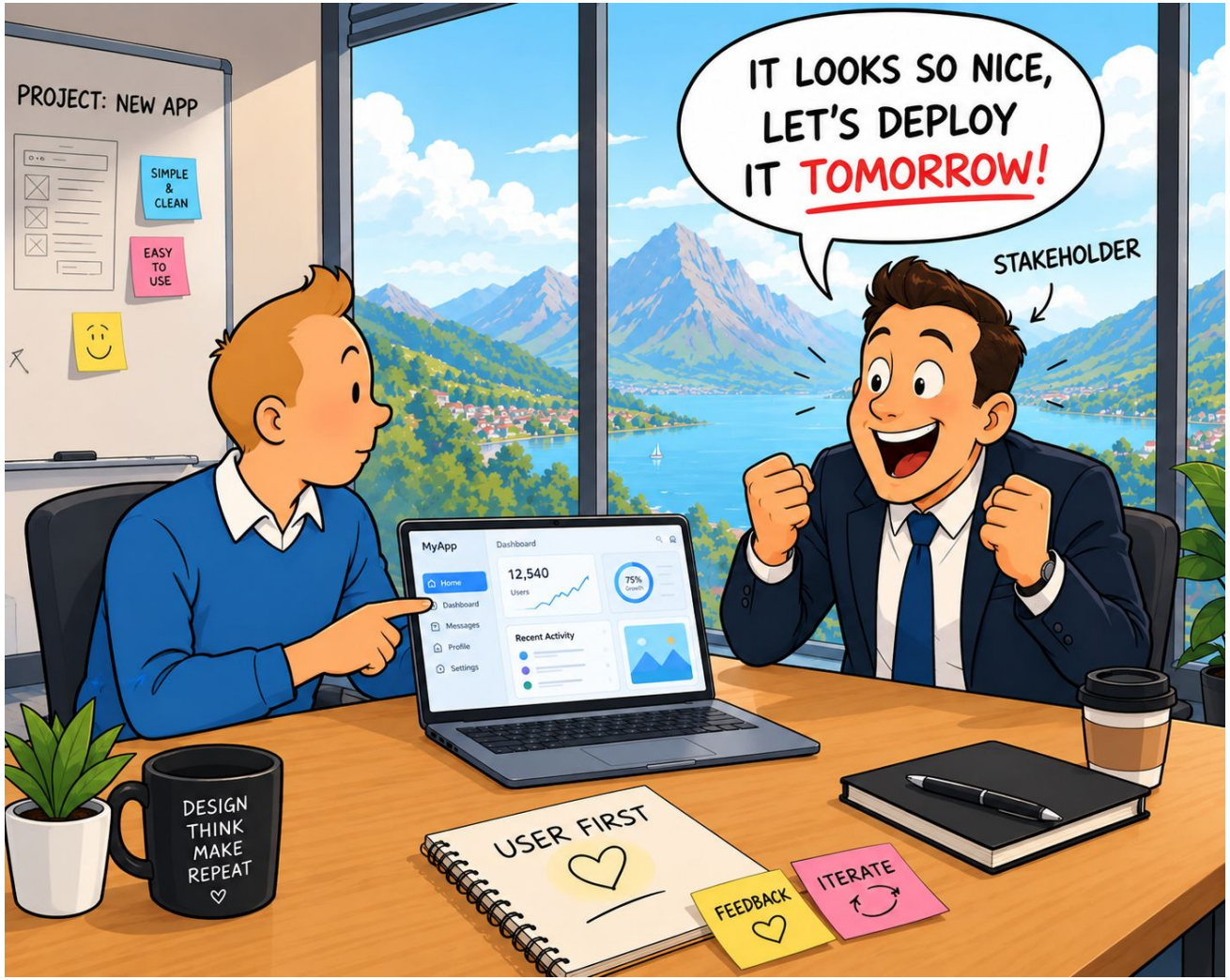
In 2026 (with AI)



Prototypes may be built with Claude Design, GitHub Copilot, Lovable, Replit, etc

2026: presenting prototype to a stakeholder

- Before showing your prototype, we **state expectations** that a prototype is just a mockup
- No backend, no database, etc
- It is just part of the ideation process



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Providing context

1 Knowledge

Stakeholder inputs & domain knowledge, converted to Markdown

Knowledge documents

- Call transcripts
- Specification docs
- Slide decks
- Emails
- ...

2 Specifications

Structured, complete and ready for the team

PRD

1. Overview
2. Goals
3. User stories
4. Requirements
5. Open questions
6. References

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

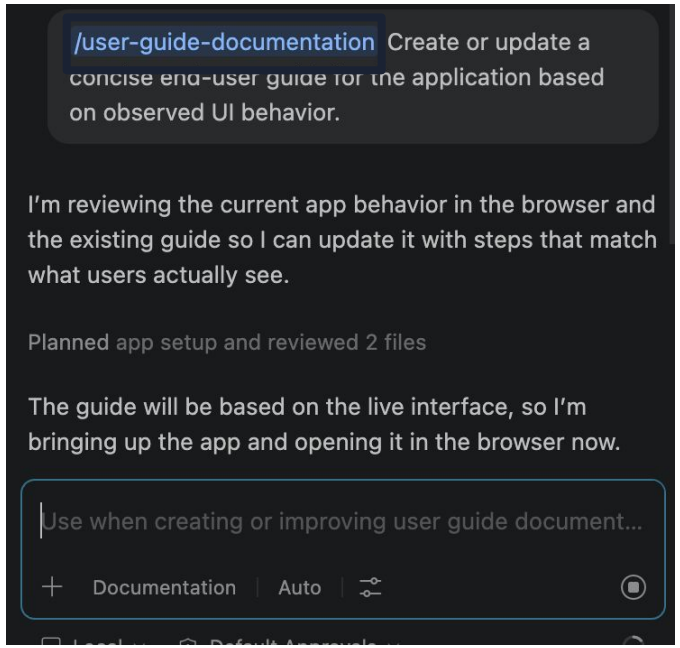
Working with Skills

- They allow to create your own contextual instructions
- Load them on the fly using a prompt
  - Example: /user-guide-documentation

```
---
name: user-guide-documentation
description: "Use when creating or improving end-user
documentation for a feature, workflow, or product
page. Helps draft clear, step-by-step guides…"
```

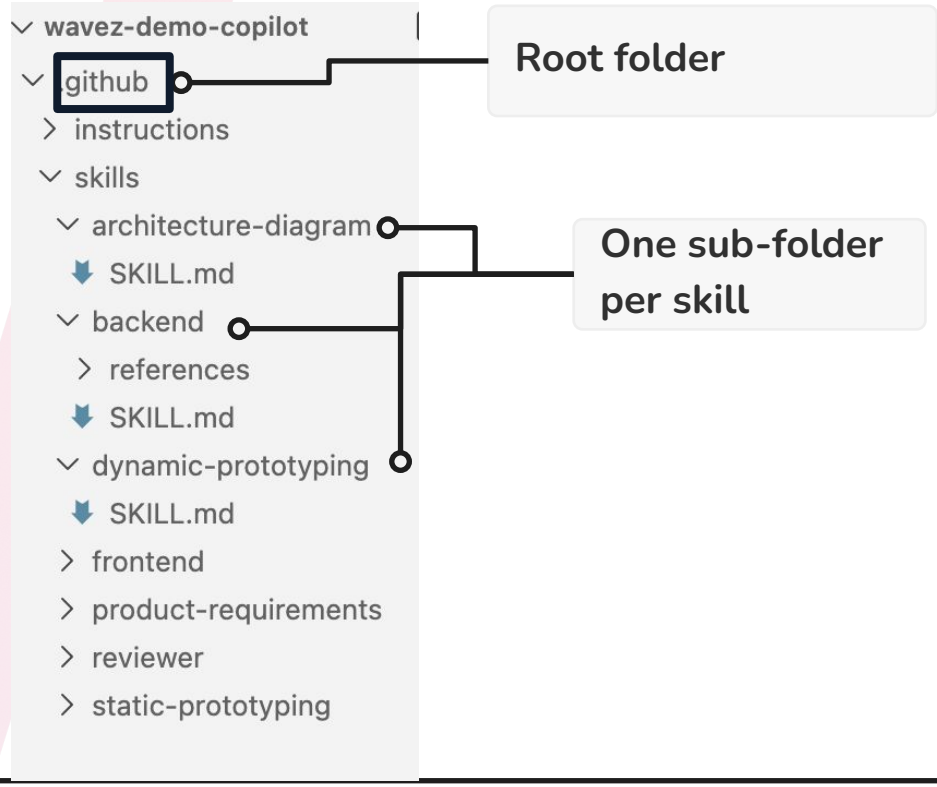
SKILL.md

```
user-guide-documentation/
├── SKILL.md    # Required: instructions
├── scripts/    # Optional: executable code
├── references/ # Optional: documentation
```

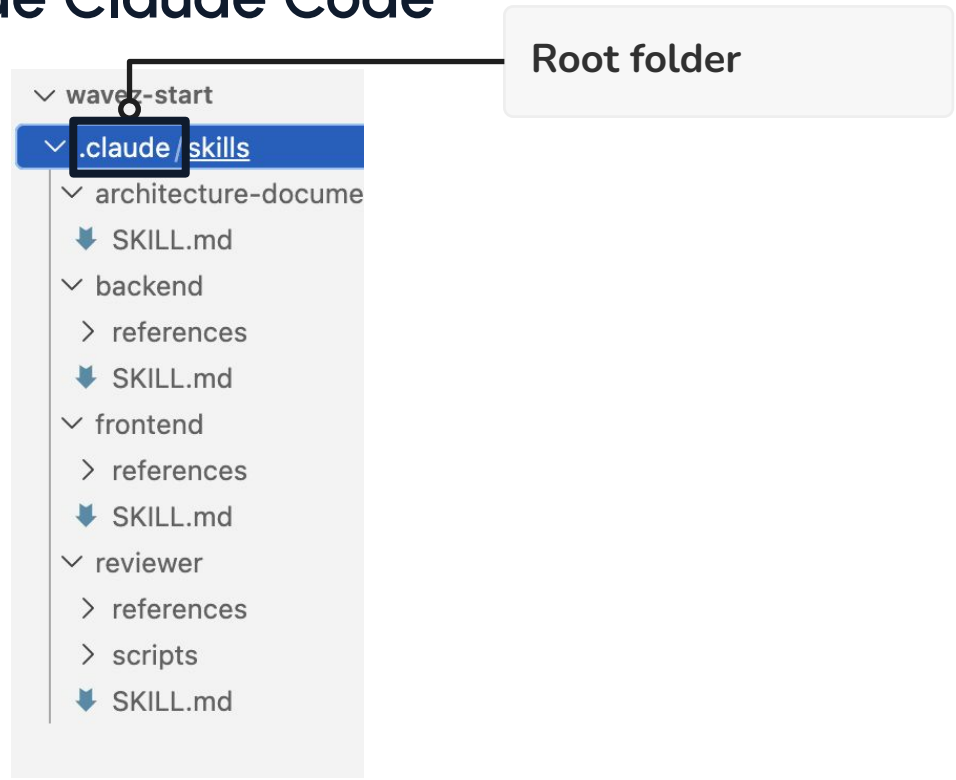


Storing Skills inside your workspace

Inside GitHub Copilot



Inside Claude Code



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Should you update your skills?

- Yes! Skills are living artefacts, they are meant to be updated
- wavez.org provides a quickstart. Skills are meant to be customised for your project

ℹ️ **In the next major release, Wavez will provide a way to update skills automatically. Stay tuned!**

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

1 Specs

Event management · you are the product manager · 45 min

a	b	c	d	e	f
Open the workspace	Get the skills from Wavez	Requirements brief	PRD	Dynamic prototype	Feedback, PRD v2

From stakeholder notes to a PRD

c. Brief

- Only what the notes say: "do not invent requirements"
- The gaps become open questions (8 at most)
- The brief is the source of truth from now on

d. PRD

- The skill asks you questions first
- Answer from the brief; when it is silent, decide and say so: "(decision)"
- Check: buttons and modal on the existing page, one rule per field, criteria on every story

An acceptance criterion

Given the "Add event" modal,
when I click Save with no start date,
then "Start date and time is required." shows under the field and nothing is saved.

Testable through the screen: QA turns it into a test case in Section 2.

Test the PRD with a clickable prototype

A problem found now costs a PRD edit. Found after implementation, it costs code.

eventStore.js: data and rules › **index.html: same conversation** › **Assumptions → prototype-notes.md**

Test the five scenarios

- Same look as current-app/listing-page.png, plus two buttons
- Add "Testing with Playwright" on the 5th of next month
- Save with no start date: refused, message under the field
- Embargo after the start date: refused, with the reason
- Edit "Contract Law for Speakers", venue "Funan Level 7"; Escape closes



Mind the dates

- The sample events are dated March 2026: they show as already started
- Use future dates for the events you add
- No past dates? Decide what happens when an old event is edited

Close the loop with feedback

Claude plays the Events Coordinator
A role-play does not replace a real user, but finds the obvious gaps first.

Show, do not tell
Two screenshots: the page with the buttons, the modal with several errors.

Pick assumptions that matter
Ones that change what the coordinator sees or can do.

PRD v2: check both ways

- Every change in CHANGES traces back to feedback, an assumption or a test finding.
- Every feedback point appears in the PRD: a requirement or an open question.

Then commit: git add . and git commit -m "Event management PRD"

2

Implementation

Event management · you are the developer, then QA · 85 min

a	b	c	d	e	f	g
Run the app	Dev and QA skills	PRD handover, gap analysis	Backend	Frontend	Review	Acceptance tests

Hand over the PRD, then measure the gap

Two terminals

- Application terminal: npm run dev, keep it running
- Second terminal: every other command

The data

- npm test uses its own database
- npm run db:reset puts the seed data back

Gap analysis (Plan mode, reads only)

For each PRD requirement: implemented, partly or not at all, and which files.

You see the size of the work and what already exists before any code is written.

No PRD from Section 1? cp ../../reference/event-management-prd.md docs/product/

Plan, approve, implement, hand over

Plan in docs/plans/ › You review and approve › Implement, 2 stops › Handover: Test Result PASS

Stay in Manual

- "Plan mode" in the prompt is the skill's planning step
- It saves the plan file, which Claude Code's Plan mode would block

At each stop

- Look at the changed files
- Then reply: continue
- Backend: every PRD rule in the plan? Use the handover's Try it curl: no start date

The handover

- Endpoints, files, test result, notes for the next skill
- Frontend runs in the same conversation: it reads the API contract from it

Review with a fresh pair of eyes

/reviewer, not /code-review
Ticked as "Code review" in Wavez. Claude Code's built-in /code-review is something else.

Checked against the PRD
Its Requirements Conformance table shows each requirement as met or not, with evidence.

With both handovers
It reviews what changed and why, independently of the skills that wrote it.

APPROVE
Run npm test, commit.

REQUEST CHANGES
Fix what you agree with, or note it. Test, commit.

BLOCK
Fix the blocking issues with /backend or /frontend first.

Acceptance tests: evidence, not opinions

npm run db:reset › Plan: a test per criterion › You approve the plan › Browser run + .docx report › npm run db:reset

A test failed? Ask Claude where the problem is

- In the application: fix with /backend or /frontend
- In the PRD: the request was wrong or silent
- In the test plan: the expected result is out of date
- In the test script: selector, timing, text case



The deliverable

- *_MANUAL_TEST_PLAN.md next to the PRD
- *_Test_Execution_Report.docx in references/, with screenshots and an acceptance criteria sign-off
- Commit both: request to tested feature, done

3

Existing feature

Listing page and search · from the code to a spec · 45 min

a

Write the spec from the code

b

Implement the spec

c

Test and check for regressions

The code is the truth, SPEC.md is the intent

Read

- Current behaviour in plain English, with file references
- Every place where the code and SPEC.md disagree
- Root causes of the search problem, file and line

Decide

- Requirements LP-1, LP-2…, each traceable to the request
- Ranking disagreements stay open questions: the order must not change
- Not sure? "Show me the code"

The request

"Make the page clearer and fix the search, without changing the order in which events are ranked."

Every LP requirement traces back to this sentence. Commit the spec: it is the contract for the change.

Fix it, prove it, do not break it

Failing tests first

- Tests that reproduce the bug, and fail
- Then the fix makes them pass
- Stop to see them fail before the fix

Check by hand

- Is the page clearer?
- Search the topic Data, then the venue marina
- Counters match the cards shown
- The ranking order is unchanged

Regression UAT

- Run both plans, db:reset before each
- A failure: regression or intended change?
- Regression: fix it with /backend or /frontend
- Intended: update the old test plan and PRD, run again

What you built today

	New feature (Sections 1–2)	Existing feature (Section 3)
Specs	From the user: brief → PRD → prototype → feedback → PRD v2	From the code: request → analysis → spec with root causes
Implementation	Backend → Frontend → Reviewer	The same flow, failing tests first
Tests	Acceptance tests from the PRD	Acceptance tests from the spec + regression

Two specifications, two sources of knowledge, one workflow.

Take it back to your team

Write the spec first, let the agent apply it, and keep the evidence.



Pick your skills on Wavez
wavez.zenika.com



Try Architecture Documentation
Document the app as it is now



Questions
And your feedback on today